

Architecture — Secure Multi-Tenant Data Platform

1. System Overview

Three tenants previously shared one MongoDB, separated only by a `clientId` field and the seed data proves that design failed (14 cross-tenant counselor assignments, approx 100 successful writes by read-only client viewer roles, 11 invalid/null `clientIds`). The replacement makes cross-tenant reads impossible: each tenant lives in its own physical database, so there is no shared query path to get wrong.

One Docker image runs three processes:

- A one-shot migrator (ingests + tokenizes seed data, exits 0)
- The api (Fastify HTTP)
- A worker (BullMQ consumers + daily cron).

Docker compose gates api/worker on the migrator completing successfully — a single command brings up a fully migrated, serving system.

2. Database Schema Design

Database	Schema	Rationale
<code>tenant_<slug>__v1</code>	<code>borrowers</code> , <code>conversations</code> , <code>payments</code> , <code>audit_logs</code>	All borrowers data. One DB per tenant and versioned naming supports registry-based cutover and rollback operations.
<code>_system</code>	<code>tenant_registry</code> , <code>users</code> , <code>clients</code> , <code>revocations</code> , <code>security_incidents</code> , <code>quarantine_borrowers</code> , <code>migration_log</code> , <code>legacy_access_logs</code>	Cross-tenant platform metadata. Legacy access logs archived working as breach evidence.
<code>_vault</code>	<code>tenant_deks</code> , <code>borrower_tweaks</code>	Encryption keys, physically separate from the data they protect and touched only by the token service as a security measure.

Key fields. The `borrowers` collection stores `phoneToken`, `aadhaarToken`, `bankAccountToken`, and `panToken`, never a plaintext PII. A migration validation step makes sure that no plaintext PII remains. Indexes are defined on `borrowers` {`assignedTo`, `status`, `dpdBucket`}, `conversations` {`borrowerId`}, and `payments` {`borrowerId`, `reference`}.

No `clientId`. `clientId` is deliberately absent from tenant collections ,the database is the tenant.

Connections. A single `MongoClient` is shared by all databases, because MongoDB databases are namespaces, not connections. This satisfies the 10-connection budget: migrator uses 8 (runs alone), then `api(6) + worker(4) = 10`. A client-per-tenant design would breach the cap at 3 tenants and collapse at 10+.

Registry. `_system.tenant_registry` maintains the mapping `clientId` → { `dbName`, `version`, `status` }. The router resolves the active database through a cache with a 30-second TTL. During re-migration, the registry pointer can be switched atomically, enabling zero-downtime cutovers and rapid rollback when needed.

3. Tenant Isolation

Every request passes a middleware pipeline: requestId → authn (JWT + revocation check) → authz (deny-by-default policy matrix) → tenant context → handler → audit. Tenant resolution depends on role:

- **Tenant-bound roles (debt-counselor, client-viewer):** The tenant ID is taken directly from the user's JWT token. Users cannot provide or change the tenant through API requests, so they can only access their own tenant's database.
- **Cross-tenant roles (admin, engineer):** These users can choose a tenant using the X-Tenant-Id header, which is checked against the tenant registry before access is granted. For operations that span all tenants, the system reads from each tenant database through the registry. Every access is recorded in the corresponding tenant's audit log.

Single choke point: handlers never name a database. They call getTenantDb(), which reads tenantId exclusively from AsyncLocalStorage. One auditable function maps tenant → DB handle, covered by a concurrent-load isolation test (3 tenants × parallel requests, zero ID overlap).

Breach response: a tenant-bound user sending X-Tenant-Id ≠ JWT tenant gets an immediate token revocation plus a security incident flagged for review. Counselor scope is enforced in the data layer: lists are constrained to { assignedTo: userId } inside the query, and a by-ID fetch of an unassigned borrower returns a 403.

Resilience: a per-tenant circuit breaker at the router opens after 5 consecutive failures (30s), fast-failing that tenant with a scoped 503 while other tenants keep serving.

4. PII Tokenization

Scheme: NIST FF3-1 format-preserving encryption with a per-tenant DEK and a per-borrower tweak. The master key (env secret) wraps each tenant DEK with AES-128-GCM in _vault. FF3-1 satisfies all four token requirements simultaneously:

- **Format-preserving:** phone stays 10 digits, Aadhaar 12, bank account 9–18; PAN (AAAAA9999A) is segment-wise FF3; email keeps @/domain shape; names keep letters and spaces (FF3 radix-26 per word, keyed substitution for short runs).
- **Consistent:** FF3-1 is deterministic — same value + same tweak ⇒ same token. A borrower's phone tokenizes identically in their record, transcripts, and payments, with no lookup table as token = FF3_1(Key, Tweak, Plaintext)
- **Reversible:** it is encryption; the response shaper detokenizes for authorized roles only.
- **Tenant-scoped:** DEKs are per tenant, so the same phone in Sunrise and Metro yields different tokens.

Conversation text: Some existing conversation messages contain plaintext phone numbers and other PII. During migration, known borrower PII values are replaced with their corresponding tokens inside message text. Only verified borrower PII is tokenized, preventing accidental replacement of unrelated numbers such as payment references or amounts.

Read pipeline: data is tokenized at rest; reads pass through a shaper that detokenizes, masks (last-4), or omits per role: admin = full; counselor = phone/name clear, Aadhaar/PAN/bank/email masked (nothing was given about emails so i took the decision of masking them); engineer = all masked; client-viewer = aggregates only, no individual rows. Unauthorized fields are omitted, not masked.

Erasure: Each borrower has a unique tweak. If a borrower requests deletion, destroying that tweak makes their tokenized data permanently unrecoverable without affecting any other borrower or the tenant's encryption key. In addition, a daily cleanup job removes PII for accounts that have been closed for more than 90 days.

5. Tenant Context Through Async Paths

AsyncLocalStorage (ALS) keeps important request information such as `tenantId`, `userId`, `role`, `scope`, `jti`, `requestId`, `maskingLevel` available throughout the processing of a request, so components like the database router, token service, and audit logger can access it directly without passing it as function arguments. However, ALS does not automatically work across message queues or background jobs. Therefore, whenever a job is added to a queue, the `tenantId` is included in the job payload, and the worker restores the ALS context when processing the job to ensure all operations run under the correct tenant.

Async path	Tenant source	Context re-established by
Background job (compliance report)	<code>job.data.tenantId</code> set by trusted enqueueing code	worker wraps the processor in <code>runWithContextAsync(ctx, fn)</code>
Scheduled task (overdue check, erasure)	cron dispatcher enqueues one job per registry tenant	each per-tenant job runs in its own ALS context — never one global pass
Webhook (payment gateway)	parsed from reference prefix (<code>PAY-client_s-...</code>), validated against registry	unknown prefix → audited rejection; payment/borrower tenant mismatch → incident

Because the audit writer reads tenant from ALS, jobs and webhooks log into the correct tenant's `audit_logs` automatically. One-job-per-tenant fan-out also isolates failures: a down tenant fails its own job; the daily run for the others still completes.

6. Tradeoffs

- **A single shared MongoClient:** One `MongoClient` is shared across all tenant databases instead of creating a separate client for each tenant. Tenant isolation is enforced through separate databases and request routing, while staying within the 10-connection limit.
- **A single MongoDB instance:** All tenants are hosted on the same MongoDB server rather than separate clusters. This keeps connection usage low and fits within the connection budget. Separate clusters can be added later if stronger infrastructure-level isolation is required.

Environment and config validation use `Zod` in `src/config/env.ts`. We chose `Zod` over hand-rolled checks:

	With Zod	Without Zod
Schema style	Declarative, concise schema	More boilerplate helpers
Types	<code>z.infer</code> keeps types in sync with validation	Manual interface + validation can drift
Errors	Good error messages out of the box	You write error messages yourself
Dependencies	Extra dependency (small bundle)	Zero extra deps

For a small `env` surface (Mongo URI, Redis, JWT secret, master key) the dependency cost is worth failing fast at startup with typed config shared across `api`, `worker`, and `migrator`.